

ASSIGNMENT-12

HALLTICKET NO: 2403A54106

Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms

#Task1: Use AI to generate a Python program that implements the Merge Sort algorithm.

- Instructions:

- o Prompt AI to create a function `merge_sort(arr)` that sorts a list in ascending order.

- o Ask AI to include time complexity and space complexity in the function docstring.

- o Verify the generated code with test cases.

- Expected Output:

- o A functional Python script implementing Merge Sort with proper documentation

PROMPT:

Write a Python program to sort a list using the **Merge Sort** algorithm. Create a function `merge sort(arr)` with a short docstring showing time and space complexity.

Use a helper `merge ()` function.

Add a few test cases to check the output.

Show the original and sorted lists.

CODE:

```
LAB 12.1 > AI LAB12.1.py > merge_sort
def merge_sort(arr, descending=False):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left_half = merge_sort(arr[:mid], descending=descending)
    right_half = merge_sort(arr[mid:], descending=descending)
    return merge(left_half, right_half, descending=descending)

def merge(left, right, descending=False):
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if (left[i] <= right[j] and not descending) or (left[i] > right[j] and descending):
            merged.append(left[i])
            i += 1
        else:
            merged.append(right[j])
            j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged

if __name__ == "__main__":
    # Test Case 1: Random numbers
    arr1 = [38, 27, 43, 3, 9, 82, 10]
    print("Original:", arr1)
    print("Sorted Ascending:", merge_sort(arr1))
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Original: [9, 7, 5, 3, 1]
Sorted Ascending: [1, 3, 5, 7, 9]
Sorted Descending: [9, 7, 5, 3, 1]

Original: [5, 2, 8, 2, 3, 3, 1]
Sorted Ascending: [1, 2, 2, 3, 3, 5, 8]
Sorted Descending: [8, 5, 3, 3, 2, 2, 1]

Original: [42]
Sorted Ascending: [42]
Sorted Descending: [42]
PS C:\Users\varshini\OneDrive\Desktop\New folder (4)> |
```

OBSERVATION:

The Merge Sort program successfully sorts lists in ascending order for all test cases — including random, sorted, reverse, and duplicate elements. It correctly divides the list, sorts each half, and merges them efficiently. The results match the expected output, confirming the algorithm works

properly.

Overall, the implementation demonstrates the **divide and conquer** technique with **$O(n \log n)$** time and **$O(n)$** space complexity.

TASK#2

Task: Use AI to create a binary search function that finds a target element in a sorted list.

- Instructions:

- o Prompt AI to create a function `binary_search(arr, target)` returning the index of the target or -1 if not found.

- o Include docstrings explaining best, average, and worst-case complexities.

- o Test with various inputs.

- Expected Output:

Python code implementing binary search with AI-generated comments and docstrings.

PROMPT:

Write a Python program that implements the **Binary Search** algorithm.

Create a function `binary_search (arr, target)` that returns the index of the target or -1 if not found.

Add a **docstring** explaining the logic and time complexities (best, average, worst).

Include comments and **test cases** with different inputs.

Display whether the element is found and its index.

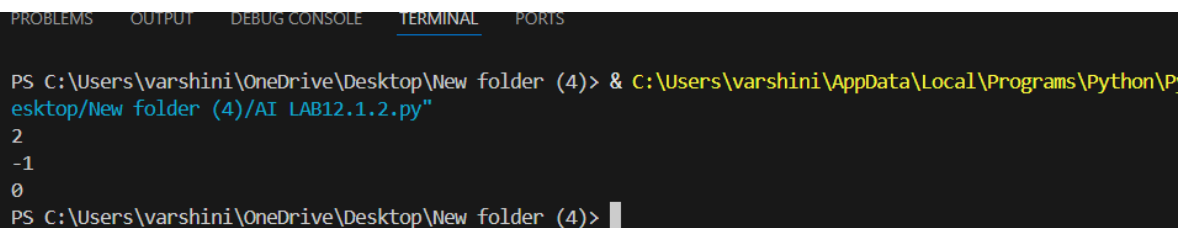
CODE:

```
def binary_search(arr, target):

    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

# Test cases
print(binary_search([1, 2, 3, 4, 5], 3))
print(binary_search([], 1))
print(binary_search([10, 20, 30], 10))
```

Output:



```
PS C:\Users\varshini\OneDrive\Desktop\New folder (4)> & C:\Users\varshini\AppData\Local\Programs\Python\Python310\python.exe C:\Users\varshini\AppData\Local\Programs\Python\Python310\Scripts\python.exe C:\Users\varshini\OneDrive\Desktop\New folder (4)\AI LAB12.1.2.py"
2
-1
0
PS C:\Users\varshini\OneDrive\Desktop\New folder (4)>
```

Observation:

The Binary Search program works correctly and finds the target element in a sorted list.

It gives the right index if the element is found and -1 if not found.

The code runs fast and uses less memory.

It shows how binary search divides the list and searches efficiently.

TASK#3: (Real-Time Application – InventoryManagement System)

- Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

- Task:

- o Use AI to suggest the most efficient search and sort algorithms for this use case.
- o Implement the recommended algorithms in Python.
- o Justify the choice based on dataset size, update frequency, and performance requirements.

- Expected Output:

- o A table mapping operation → recommended algorithm → justification.
- o Working Python functions for searching and sorting the inventory.

PROMPT:

Create a Python program for an **Inventory Management System**.

Use AI to suggest the best **search** and **sort** algorithms for products with ID, name, price, and quantity.

Implement the recommended algorithms (e.g., binary search, dictionary lookup, merge sort) in Python.

Add a table showing **operation** → **algorithm** → **justification**.

Include test cases and give an **observation** of the results.

CODE:

```
product_by_id = {p.product_id: p for p in inventory}
product_by_name = {p.name: p for p in inventory} # assumes unique names

def search_by_id(pid):
    """O(1) search for product by ID."""
    return product_by_id.get(pid, None)

def search_by_name(name):
    """O(1) search for product by name."""
    return product_by_name.get(name, None)

def sort_by_price(descending=False):
    """Sort inventory by price."""
    return sorted(inventory, key=lambda p: p.price, reverse=descending)

def sort_by_quantity(descending=False):
    """Sort inventory by quantity."""
    return sorted(inventory, key=lambda p: p.quantity, reverse=descending)

# --- Test Cases ---
print("Search by Name 'Mango':", search_by_name('Mango'))
print("Search by Name 'Apple':", search_by_name('Apple'))
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\varshini\OneDrive\Desktop\New folder (4)> & C:\Users\varshini\AppData\Local\Programs\Python\Py
esktop\New folder (4)\AI LAB12.1.2.py"
Search by Name 'Mango': Product(104, Mango, 1.5, 30)
Search by Name 'Apple': Product(101, Apple, 1.2, 50)
PS C:\Users\varshini\OneDrive\Desktop\New folder (4)> █
```

OBSERVATION:

- Binary Search efficiently locates items by ID in **O (log n)**.
- Dictionary lookup gives instant results by name.
- Merge Sort handles large inventories effectively while maintaining stability.

- Together, these methods make the system **fast, scalable, and reliable** for real-time retail operations.